

## Editorial: Scales

This is based on the well-known problem of ordering five coins with 7 (binary) weighing. There are 120 permutations and 128 possible sequences of seven answers, and it is indeed possible to find a strategy which always works.

In our problem, there are 720 possible answers, and with at most 6 ternary weighing, theoretically we could obtain  $3^6 = 729$  possible answers. The gap is very small, but it turns out that it is again possible to find a strategy which uses just 6 weighing.

The easiest solution here is to generate a memorization function which, for each possible subset of possible permutations, tries all the possible questions, and returns the best one. It is possible to bound the branching, as follows:

- Always take care that there are at most 243 consistent results after 1 weighing, at most 81 consistent results after 2 weighing, etc.
- If, for the given subset of power  $P$ , we have already found a solution which uses  $N$  weighing, and  $P > 3^{(N-1)}$ , then do not look further (we have already found the best possible answer).

With these optimizations, we could write a program which generates the strategy tree, in the form of a program which could be submitted for judging (see `builder.cpp`, which does not build it in the right format yet, but is close enough). With the optimizations above the builder runs fast enough, so it is also possible to generate it on the fly.

## Editorial: Teams

### No preprocessing, $O((n + m) \log n)$ per query

The following greedy assignment works: take the smallest team size  $k$  and repeat  $k$  times: assign the child with  $a_i \leq k$  and smallest  $b_i \geq k$ . It can be implemented in time  $O((n + m) \log n)$  by processing the teams and children in increasing order and maintaining a priority queue of children available for assignment.

### Constructive approach

Now we enter the world of geometry. If we map the children to points  $(a_i, b_i)$ , forming a set  $P$ , a team of size  $k$  can be mapped to a rectangle  $R(k) = [0, k] \times [k, +\infty]$ . We can reformulate our query as follows: - is it possible to simultaneously assign  $k_i$  points to a rectangle  $R(k_i)$ ? (no point can be assigned to 2 or more rectangles)

From now on we assume that we preprocess the set  $P$ , so that we can query the number of points in some rectangle  $[x_1, x_2] \times [y_1, y_2]$  in time  $O(\log n)$ . This can be done for example using a persistent segment tree or some other method.

We again try to form teams in the order of increasing  $k$ s. The idea is to maintain the set of "forbidden" points, that is, the region of plane containing the points which are either assigned to some previously processed rectangle, or not available because  $y_i < k$ .

We need to assign  $k$  lowest points (smallest  $y$ ) from outside the forbidden region and update the forbidden region. If we represent the region by corners  $c_1, c_2, \dots, c_z$ , the update will remove some (maybe 0) of consecutive corners starting before  $c_2$  and insert a single new corner.

Therefore, we only need to find the right place for the new corner.

### $O(n \log n)$ preprocessing, $O(m^2 \log n + m \log^2 n)$ query

We can find the appropriate corner  $c_l$  such that there are at least  $k$  allowed points in a rectangle  $[0, k] \times [0, c_l]$  in time  $O(l \log n)$  (we test sequentially  $c_1, c_2, \dots$ , each time asking about points in some rectangle and summing them up).

Once we found  $c_l$ , we know that the  $y$  coordinate of our new corner will be somewhere between  $y(c_{l-1})$  and  $y(c_l)$ . We can binary search for the exact  $y$  in time  $O(\log^2 n)$ .

### $O(n \log n)$ preprocessing, $O(m \sqrt{n} \log n)$ amortized query

We can combine the two solutions above to obtain a better bound: - we run the first solution if  $m > \sqrt{n}$ , - we run the second solution if  $m \leq \sqrt{n}$ .

### $O(n \log n)$ preprocessing, $O(m \log^2 n)$ query

If we store the corners in some kind of binary search tree and decompose the forbidden area into rectangles (picture) so that each corner knows how (is responsible for) many points are there in its rectangle, then we can binary search for the  $y$  of new corner directly: each guess will involve a range sum in the corners structure and a single query to the points structure.

Thus, the running time is  $O(m \log^2 n)$ .

### Non-constructive approach

The above solutions, although implicitly, construct the assignments. However, our question is binary and thus another approach is possible.

The Hall theorem says the following:

It's not possible to assign children to teams if there exists such subset A of team sizes  $k_i$ , that the set of points that can be assigned to any team from A (let's call those points neighbors) is smaller than the sum of numbers in A.

Therefore, we want to construct such set A, that the number  $c(A) = |\text{neighbors of A}| - (\text{sum of numbers in A})$  is smallest possible. If the smallest  $c(A)$  turns out to be negative, then the answer is NO, otherwise it's YES.

Assume that  $k_i$ 's are distinct and sorted (just for clarity).

We can propose a simple dynamic programming solution.

Let  $D_i$  be the minimal  $c(A)$  such that  $k_i$  is the greatest element of A. Then:  $D_i = \min \{ D_j + Z(j, i) - k_i : j < i \}$ , where  $Z(j, i) = |\text{children } [a, b] \text{ s.t. } a \in [k_j + 1, k_i], b \geq k_i|$

Optimal  $c(A)$  is thus equal to  $\min \{ D_i \}$ .

## Another $O(m^2 \log n)$ solution

As computing  $Z(j, i)$  is asking about number of points in some rectangle, we can implement this DP in  $O(m^2 \log n)$ .

## Another $O(m \sqrt{n} \log n)$ amortized solution

This can be again combined with the brute force solution to speed it up.

## $O(m \log n)$ solution

Let us assume, that we have three indices  $i < j < k$ , such that it's more beneficial to take index  $i$  than  $j$ , while computing the minimum in the formula for  $D_k$ . Then, for any  $l > k$ , it's also more beneficial to take index  $i$  instead of  $j$ . Why? We have from our assumption:

$D_i + Z(i, k) \leq D_j + Z(j, k)$ , which is equivalent to:

$D_j - D_i \geq |\text{children } [a, b] \text{ s.t. } a \in [k_i + 1, k_j], b \geq k_k|$ .

If we replace  $k_k$  with  $k_l \geq k_k$ , the right side won't increase, so indeed  $D_i + Z(i, l) \leq D_j + Z(j, l)$ .

Therefore, for any indices  $i < j$  we can compute the exact moment  $W(i, j)$  of DP computation, when the index  $i$  will be better than  $j$ .

It can be done in time  $O(\log n)$ .

The improved DP goes like this: when computing  $D_k$ , we maintain a set of those indices that might be useful according to our current knowledge. It also means that if indices  $i$  and  $j$ ,  $i < j$  are in the set right now, then  $j$  is more beneficial. Hence, the last index from the set constitutes an optimal transition for  $D_k$ .

Maintaining the set of indices involves a queue of events. For each two indices  $i < j$  that happen to be neighbors in the set at some point of time, we push the event "remove  $j$  from the set once you reach computation of  $D_l$ ", for some  $l$ .

There are  $O(m)$  set updates/accesses and each time we use  $O(\log n)$  time to compute a moment when  $j > i$  is useless.

It remains to show, how to preprocess the input points to be able to find  $W(i, j)$  in time  $O(\log n)$ . Let  $B = D_j - D_i$ .

We need to find smallest  $y$ , such that there are at least  $B$  input points in  $[k_i + 1, k_j] \times [y, +\infty]$ . This can be solved with a variant of a segment tree. However, in a node  $[A, B]$  of the segment tree we store all the points (children) with  $y$  in  $[A, B]$ . The points inside a single node are sorted by increasing  $x$  and each point stores the pointers to:

- the first point with  $x' \geq x$  and last with  $x' \leq x$  in  $[A, \text{mid}]$
- the first point with  $x' \geq x$  and last with  $x' \leq x$  in  $[\text{mid} + 1, B]$ .

This allows us to binary search for  $k_i$  and  $k_j$  only in the root of the segment tree, and then just follow the pointers on a path to the leaf.

## Editorial: Boxes

The problem has a solution in linear time. The solution is basically greedy, and it is based on several observations. Discovering only a subset of those observations usually leads to a correct but slower solution.

We will use the word *trip* to denote the part of a solution between two visits to a warehouse. The numbering of locations increases in the CW direction. (Imagine the circle as a clock face with the warehouse at 12 = 0, and locations at 1 through 11.)

Clearly, in an optimal solution we will never go twice along the same segment in the same direction during the same trip – any such trip can be shortened while still visiting the same set of locations. The moment when we deliver boxes also do not matter. WLOG we may assume that we deliver a box we want the first time we visit its location. Hence, we can divide the trips into three groups:

*CW trips:* The boxes are delivered while going CW, we return by going CCW.

*CCW trips:* The same with directions reversed.

*Full-circle trips:* The entire circle is traversed once in either direction.

Also, we may easily note that each CW/CCW trip must end at one of the boxes we leave in that trip – going any farther can be skipped.

**Lemma 3** *In an optimal solution, the distance traveled CW in a CW trip is at most  $l/2$  (half of the circle). The same holds for CCW trips.*

The proof is trivial: if you travel more than  $l/2$  while delivering the boxes and now you want back, it is cheaper to continue in the same direction than to return – i.e., to make a full-circle trip instead.

**Lemma 4** *There exists an optimal solution satisfying the property from Lemma 3 where in each trip we deliver a contiguous subset of boxes (i.e., boxes with no other box located between them).*

Proof: Consider any optimal solution.

First of all, note that in each half of the circle, the boxes that are delivered during full-circle trips are all at least as far from the warehouse as the boxes delivered in CW/CCW trips.

This is because we could take the farthest box delivered in a CW/CCW trip and swap it for one that is closer but delivered in a full-circle trip. The full-circle trip will remain the same length, the CW/CCW trip will now be shorter, which contradicts the optimality of the original solution.

Now we know that the set of boxes delivered by all full-circle trips is contiguous. As we can pick any of them during each full-circle trip, we can easily plan the full-circle trips so that each of them delivers a contiguous subset of these boxes.

Now let us look at the boxes delivered during the CW trips. Obviously, the  $k$  farthest from the warehouse can be delivered in the same trip – this can be proved using a switching argument similar to the one above. QED.

**Lemma 5** *There is always an optimal solution with at most one full-circle trip. Additionally, we deliver exactly  $k$  boxes during that trip (or all of them if  $n < k$ ).*

Proof: It always makes sense to drop as many boxes as we can during a full-circle trip, because we can move boxes from CW/CCW trips to full-circle trips at no cost.

As we already know, there is an optimal solution in which each trip drops a contiguous segment of boxes. For any partition of boxes into contiguous segments, at most one such segment contains boxes on both sides of the circle. Only that one segment needs to be delivered in a full-circle trip, for each other segment of boxes a CW/CCW trip is at most as expensive as a full-circle one.

**Theorem 6** *The problem can be solved in  $O(n)$  time.*

Proof: Let  $l_i$  be the optimal distance to deliver the first  $i$  boxes in the CW direction using CW trips. These values can be precomputed in  $O(n)$  time. Let  $r_i$  be the same values for the CCW direction.

There may be no full-circle trip. In  $O(n)$ , try all possibilities for how many boxes are delivered in CW trips. (This is overkill but the benefit is that we do not have to handle boxes opposite the warehouse as a special case.) Look for the minimum of  $l_i + r_{n-i}$ .

There may be one full-circle trip. Again in  $O(n)$ , try all possibilities for the  $k$  boxes delivered during such a trip. Look for the minimum of  $l_i + 1 + r_{n-k-i}$ . Return the minimum of all those values.

Another proof: If there are fewer than  $k$  boxes in the location opposite the warehouse, there are clearly  $O(k)$  different possibilities for the segment delivered during a full-circle trip, and  $O(k)$  possibilities to consider if there is no full-circle trip.

If there are  $k$  or more boxes in the location opposite the warehouse, there is an optimal solution with no full-circle trip. Additionally, we can obviously assume that less than  $k$  of those boxes are delivered in CW trips. This gives us, again,  $O(k)$  possibilities for the split between boxes delivered in either direction.

In either case, we have  $O(k)$  possibilities to try. For each of those  $O(k)$  possibilities, we can compute the optimal total cost without any precomputation: we simply go through the locations of the remaining boxes with step  $k$  and add those distances up. Thus, each single possibility can be evaluated in  $O(n/k)$ , which gives a total time complexity of  $O(n)$  again.

## Editorial: Towns

The graph given in this task is an edge-weighted tree  $T$ , in which the degree of each internal node is at least three. Let  $n$  denote the number of leaves of  $T$ . The goal is to find the centers of the tree and determine if any center is also a leaf median (an internal node such that after removing the node, every component contains at most  $n/2$  leaves. In this task, the tree is not given, and the contestants must solve the task by using limited number of queries for distances between leaves. An algorithm using  $7n/2$  queries is sketched as follows. The details are in the next sections. \* First, find the centers by at most  $2n-3$  queries; and then \* for each center (at most two centers), determine if it is also a leaf median by using no more than  $3n/2$  queries.

### Radius and centers

The diameter of a tree can be found with  $2n - 3$  queries as follows. \* Farthest to Farthest: Pick an arbitrary vertex  $v$  and find a vertex  $s$  farthest to  $v$ . Find a vertex  $t$  farthest to  $s$ . It can be shown that  $d(s,t)$  is a diameter of the tree. \* Any center must be one the  $vs$ -path. \* Then the vertices on the  $vs$ -path with its distance to  $s$  closest to  $d(s,t)/2$  are centers.

### Determining if a center is also a median

Let  $v,s$  be the two leaves in the above process and  $m$  be a center on the  $vs$ -path with  $d(s,m) = r$ . We need to determine if each component of  $T - m$  has at most  $n/2$ . Let  $S$  be the set of all leaves.

First we compute the multiset  $B = \{(d(u,s) + d(s,v) - d(u,v))/2 \mid \forall u \in S\}$ . Each of the different values in  $B$  identifies a unique internal vertex on the  $sv$ -path. Let  $m^0$  be the internal vertex on the  $sv$ -path with  $d(s,m^0) = \alpha$ , where  $\alpha$  is the median of the multiset  $B$ . If we root  $T$  at the  $sv$ -path, the leaf median must be in the subtree rooted at  $m^0$ . Therefore if  $r \neq \alpha$  (i.e.,  $m \neq m^0$ ), then  $m$  is not a median. Note that there are two medians in  $B$  if  $|B|$  is even. Otherwise it remains to solve the “giant component” problem: checking if there is a component in  $T-m$  with more than  $n/2$  leaves.

Let  $X = \{u \in S \mid d(u,s) + d(s,v) - d(u,v) = 2r\}$  which is the set of the leaves branching from  $sv$ -path at  $m$ . For  $x_1, x_2 \in X$ , we have that  $x_1, x_2$  are in the same component of  $T-m$  iff  $d(s,x_1) + d(s,x_2) - d(x_1,x_2) > 2r$ . Then, we can solve the giant component problem by algorithms for the following problem: There are  $n$  color balls and we want to determine if there are more than  $n/2$  balls of the same color.

The cost is counted by the number of performed queries  $R(u,v)$  which returns Equal/NotEqual according to if  $u$  and  $v$  are of the same color.

It was shown that  $d3n/2e - 2$  queries are necessary and sufficient in Fischer and Salzberg (1982), Solution to problem 81-5, J. Algorithms, pp. 376-379. The following is another similar approach. Initially each element is itself a set. At each iteration, we arbitrarily pair up the survival sets and compare their representatives. If they are equal, then join the two sets into their union. Otherwise, mark both dead. In the case that the number of alive sets is odd, mark anyone dead. Repeating this process, eventually either there is exactly one survival set or all sets are dead. It can be shown that if there is a majority originally, then it must be the survival one. So the remaining work is to compare the survival representative with the representatives of all DEAD sets.

The number of comparisons: Let  $A_i$  denote the number of alive sets in the  $i$ -th iteration ( $A_0 = n$ ). The number of comparisons to obtain the only survival set is  $1/2(A_0 + A_1 + A_2 \dots)$ . In the second stage the number of comparisons is the number of dead sets. Let  $D_i$  denote the number of sets die at iteration  $i$ . Then, we have  $D_1 = A_0 - 2A_1$ . Similarly  $D_i = A_{i-1} - 2A_i$ . Therefore, the total number of dead sets is  $(A_0 - 2A_1) + (A_1 - 2A_2) + \dots = A_0 - (A_1 + A_2 + \dots)$ . In summary, the number of comparisons is  $(3/2)A_0 - (1/2)(A_1 + A_2 + \dots) < 3n/2$ .

# Editorial: Sorting

## Subtask1

In subtask 1, Bob do not change sequence  $s$ , thus any method that can sort the sequence in increasing order will be a solution.

A bubble sort or insertion sort may cost at most  $O(n^2)$  swaps. A merge sort or quick sort may cost at most  $O(n \log n)$  swaps.

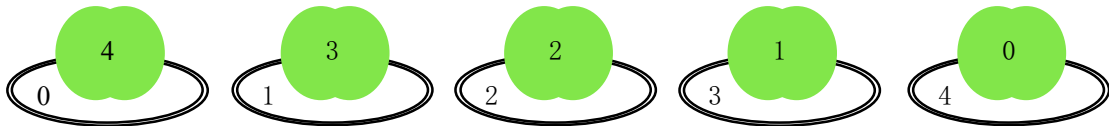
The optimized solutions is selection sort. Since the sequence contains  $0, 1, \dots, n-1$  exactly once, we can swap value  $i$  into position  $i$  one by one ( $i=0, 1, \dots, n-1$ ). This costs at most  $n-1$  swaps. It can be proved that the number of swaps is minimized.

## Subtask2

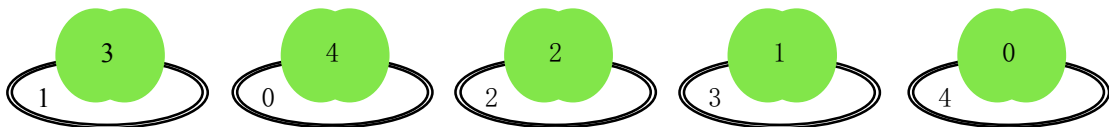
In subtask 2, Bob will swap positions 0 and 1, which makes the problem more complicated. However, the solution does not change much. We can swap value  $i$  into position  $i$  one by one from the rightmost position to the leftmost one ( $i=n-1, n-2, \dots, 1, 0$ ). The number of swaps is also minimized.

## Subtask3

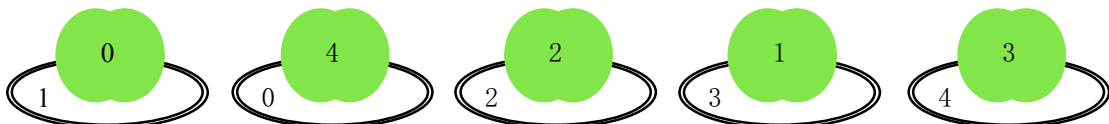
In subtask 3, we distinguish Alice and Bob's swaps. Let  $a[0]=0, a[1]=1, \dots, a[n-1]=n-1$  be  $n$  plates in a queue, and  $b[0]=s[0], b[1]=s[1], \dots, b[n-1]=s[n-1]$  be  $n$  apples on plates  $0, 1, \dots, n-1$ . Then  $s[i]=b[a[i]]$ .



If Bob swap two positions  $x$  and  $y$ , it can be viewed as two plates are swapped. For example, in the above case, if Bob swaps numbers at positions 0 and 1, the result sequence will be:



If Alice swap two positions  $p$  and  $q$ , it can be viewed as two apples  $s[p]$  and  $s[q]$  are swapped. For example, if Alice swaps numbers are positions 0 and 4, it can be viewed as apples  $s[0]=3$  and  $s[4]=0$  are swapped:



Now Bob swaps two plates in each round and Alice swaps two apples. Since they operate on different objects, the sequence of operations can be separated. Let  $t$  be the number of rounds Alice takes. It can be viewed as Bob first swap  $t$  pairs of plates, then Alice swap  $t$  pairs of apples. Since Alice can sort all apples in increasing order in less than  $n$  swaps, the minimum number of rounds is less than  $n$ .

**Solution:** One solution for this problem would be to enumerate  $t$  from 0 to  $n-1$ , and check whether Alice can sort all apples within  $t$  swaps after Bob swap first  $t$  pairs of plates. This costs  $O(n^2)$  time and will get timeout for some test cases.

**Binary Search:** For this problem, if  $(p_0, q_0), (p_1, q_1), \dots, (p_{t-1}, q_{t-1})$  be a solution for Alice in  $t$  swaps, then  $(p_0, q_0), (p_1, q_1), \dots, (p_{t-1}, q_{t-1}), (x_e, y_e), \dots, (x_{e-1}, y_{e-1})$  must be a solution for Alice in  $e$  swaps ( $e > t$ ). Which means that if there is a feasible solution in  $t$  swaps, then there must be feasible solutions in more than  $t$  swaps. Which makes the problem solvable using binary search. The time complexity reduces to  $O(n \log n)$  and full mark is expected.

**Implementation:** The implementation of this problem may be a little bit error prone. It is recommended that programs first find out pairs of numbers (i.e. pairs of apples) to swap, then transform numbers into their positions. An inversed array that maps each number to its position is helpful.

## Comments

The  $O(n^2)$  time solution is essentially updating a set of cycles under splicing (edge insertion / deletion) operations. Some care is needed in test data making to distinguish it from  $O(n \log n)$  time solutions, perhaps the bounds can be raised to ease this process.

This solution can also be made to run in  $O(n^{1.5})$  or  $O(n \log n)$  time using BBST-like data structures (e.g. <http://contest.felk.cvut.cz/11cerc/solved/r/>). This is way more work than the intended solution though.

# Editorial: Horses

To solve 1st subtask we need just calculate our answer using dynamic programming  $dp[i][j]$  - what is maximal profit if we pass  $i-1$  days and we have  $j$  horses then we got  $j * x_i$  horses and check all number of horses that we sell today and go to the next day.

## #1 Observation

First of all let's consider to points  $i$  and  $j$ , what if we sell  $k$  horses in  $i$ -th and  $j$ -th day. And check in what day it's more preferable.

Profit of  $i$ -th day:  $x_1 * x_2 * \dots * x_i * y_i$

Profit of  $j$ -th day:  $x_1 * x_2 * \dots * x_i * x_{i+1} * \dots * x_j * y_j$

It depends on this

$y_i ? x_{i+1} * \dots * x_j * y_j$

>

<

=

so if it's > then it's profitably sell horses in  $i$ -th day

if it's < then it's profitably sell horses in  $j$ -th day

if it's = then it's no matter when we sell them in  $i$ -th day or in  $j$ -th day

From this point it's clear that it is better to sell all our horses in one day that gives us maximal profit.

Using this observation we can solve 2 subtasks.

After each query we can solve problem in  $O(n)$

## #2 Observation

If all  $x_i \geq 2$ , then after 30 days  $x_i * x_{i+1} * x_{i+2} * \dots * x_{i+29} \geq 2^{30} > 10^9$ , so position less than and equal  $n-30$  never can be optimal solution, because even if  $y_{n-30} = 10^9$  and  $y_n = 1$ ,  $2^{30} * y_n > y_{n-30}$

It's enough to check only last 30 positions

## #3 Observation

The main problem in last subtask is  $x_i = 1$ , but in that cases multiplication first  $x_i$  numbers and  $x_{i-1}$  is not change, this gives us opportunity to merge consecutive positions with  $x_i = 1$ , when we merge this positions we should take the maximal  $y_i$ . So if we do that, it's enough to check last 60 positions, because it can be no more than 30 merged 1's between 30 last positions where  $x_i \geq 2$ .

So we can store our state in some structure like set, that provide us information about current merged positions, and some structure that provide us RMQ. So solution per query will be  $\log(10^9) * \log(n)$ . Then we have to calculate answer we can use something like segment tree.

So overall complexity will be  $O(m * \log(10^9) * \log(n))$ .